

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего
образования
**«Пермский государственный национальный исследовательский
университет»**
Институт компьютерных наук и технологий

ОТЧЁТ

по лабораторной работе №1
по дисциплине «Язык программирования C++»

«Типы данных, условия, циклы и массивы в языке C++»

Вариант: нечётные задачи (1, 3, 5, 7, 9), группа 2

Выполнил:

студент группы ИТ-3-2026
Денисов Кирилл Ильич
_____ «___» _____ 2026 г.

Проверила:

преподаватель
Ракина Валерия Денисовна
Оценка: _____
_____ «___» _____ 2026 г.

Пермь 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	
1 Постановка задачи.....	
2 Структура программы.....	
2.1 Состав проекта.....	
2.2 Интерфейс и проверка ввода.....	
2.3 Соблюдение Google C++ Style Guide.....	
3 Задание 1. Методы.....	
3.1 Задача 1. Дробная часть.....	
3.2 Задача 3. Букву в число.....	
3.3 Задача 5. Двузначное.....	
3.4 Задача 7. Диапазон.....	
3.5 Задача 9. Равенство.....	
3.6 Результаты тестирования.....	
4 Задание 2. Условия.....	
4.1 Задача 1. Модуль числа.....	
4.2 Задача 3. Тридцать пять.....	
4.3 Задача 5. Тройной максимум.....	
4.4 Задача 7. Двойная сумма.....	
4.5 Задача 9. День недели.....	
4.6 Результаты тестирования.....	
5 Задание 3. Циклы.....	
5.1 Задача 1. Числа подряд.....	
5.2 Задача 3. Чётные числа.....	
5.3 Задача 5. Длина числа.....	
5.4 Задача 7. Квадрат.....	
5.5 Задача 9. Правый треугольник.....	
5.6 Результаты тестирования.....	
6 Задание 4. Массивы.....	
6.1 Задача 1. Поиск первого значения.....	
6.2 Задача 3. Поиск максимального.....	
6.3 Задача 5. Добавление массива в массив.....	
6.4 Задача 7. Возвратный реверс.....	
6.5 Задача 9. Все вхождения.....	
6.6 Результаты тестирования.....	
ЗАКЛЮЧЕНИЕ.....	
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	
ПРИЛОЖЕНИЕ А Исходный код программы.....	

ВВЕДЕНИЕ

Лабораторная работа №1 по дисциплине «Язык программирования C++» посвящена базовым конструкциям языка C++: встроенным типам данных, условным инструкциям `if` и `switch`, циклам `for`, `while` и `do-while`, а также одномерным массивам и динамической памяти.

Цель работы – получить практические навыки процедурного программирования на языке C++: оформлять решение каждой задачи в виде отдельной функции, организовывать их вызов из функции `main` через удобное пользователю меню и контролировать корректность данных, вводимых с клавиатуры.

Для достижения цели решались следующие задачи:

- реализовать по пять задач (с нечётными номерами 1, 3, 5, 7, 9) из каждого из четырёх заданий: «Методы», «Условия», «Циклы», «Массивы»;
- организовать в функции `main` двухуровневое текстовое меню для вызова всех функций;
- реализовать проверку ввода: повторный запрос значения при ошибке и корректное завершение при закрытии потока ввода;
- оформить исходный код в соответствии с руководством по стилю Google C++ Style Guide [1];
- протестировать функции на примерах из условия и на граничных случаях.

Программа разработана на языке C++ [3] (стандарт C++20) в среде Microsoft Visual Studio (проект LAB1). Дополнительно код собирался компилятором `g++` с ключами `-Wall` `-Wextra` `-pedantic` `-Wconversion` (без предупреждений) и проверялся утилитой `cpplint` [5], которая контролирует соответствие руководству Google; замечаний утилита не выдала (проверка строки об авторских правах отключена, так как руководство не требует её в новых файлах). Отчёт оформлен по ГОСТ 7.32–2017 [2].

1 Постановка задачи

Решение всех задач необходимо оформить в виде функций. В главной функции `main` требуется организовать вызов всех функций с дружелюбным интерфейсом. Если исходные данные вводятся с клавиатуры, необходимо организовать проверку ввода.

Работа состоит из четырёх заданий: работа с типами данных, условия (`if`, `switch`), циклы (`for`, `while`) и типовые алгоритмы на массивах. По варианту (нечётные задачи, группа 2) из каждого задания решаются задачи 1, 3, 5, 7 и 9 (таблица 1).

Таблица 1 – Задачи варианта и реализующие их функции

Задание	№	Название задачи	Функция
1. Методы	1	Дробная часть	Fraction
	3	Букву в число	CharToNum
	5	Двузначное	Is2Digits
	7	Диапазон	IsInRange
	9	Равенство	IsEqual
2. Условия	1	Модуль числа	AbsNum
	3	Тридцать пять	Is35
	5	Тройной максимум	Max3
	7	Двойная сумма	Sum2
	9	День недели	Day
3. Циклы	1	Числа подряд	ListNums
	3	Чётные числа	Chet
	5	Длина числа	NumLen
	7	Квадрат	Square
	9	Правый треугольник	RightTriangle
4. Массивы	1	Поиск первого значения	FindFirst
	3	Поиск максимального	MaxAbs
	5	Добавление массива в массив	Add
	7	Возвратный реверс	ReverseBack
	9	Все вхождения	FindAll

Сигнатуры в условии даны в общем виде, поэтому при переносе на C++ тип `String` заменён на `std::string`, в функции для массивов добавлен параметр-размер, а имена записаны с заглавной буквы по правилам Google C++ Style Guide [1].

2 Структура программы

2.1 Состав проекта

Проект LAB1 состоит из трёх файлов (код – в разделах 3–6 и приложении А):

- `Methods.h` – объявления всех функций с комментариями и константа `kMaxSize`;
- `Methods.cpp` – реализация функций заданий, ввода с проверкой и меню `Task1()`–`Task4()`;
- `main.cpp` – функция `main` с главным меню; в начале `main` вызов `setlocale` включает русскую локаль, а в Windows кодировка консоли переключается на UTF-8.

2.2 Интерфейс и проверка ввода

Меню двухуровневое: в главном меню выбирается задание, во вложенном – задача. Пункты 1–5 вложенного меню соответствуют задачам 1, 3, 5, 7 и 9 из условия. Пункт 0 возвращает назад или завершает программу.

Весь ввод выполняется через функцию `ReadLine`, которая считывает строку целиком. Функции `ReadInt` и `ReadDouble` разбирают строку с помощью `std::istringstream` и принимают её, только если в ней записано одно число без лишних символов. Поэтому строки «abc», «4x» или «5,25» отклоняются, и значение запрашивается повторно. Функция `ReadIntInRange` дополнительно проверяет попадание в отрезок, а `ReadCharToNum` требует ровно один символ от «0» до «9». При закрытии потока ввода (Ctrl+Z) программа корректно завершается. Работа меню и обработка ошибок ввода показаны на рисунке 1.

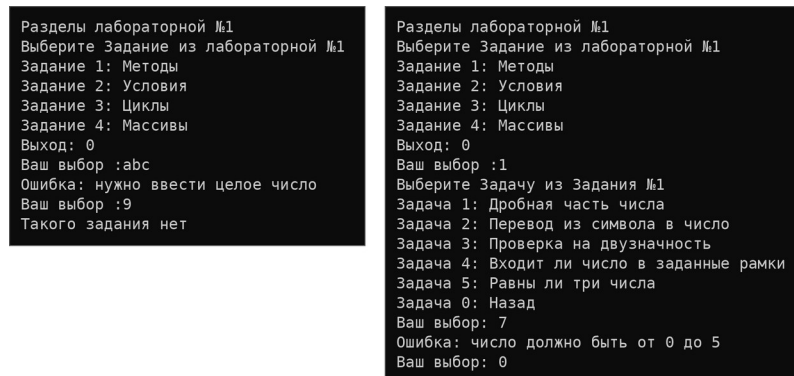


Рисунок 1 – Главное меню, меню задания 1 и обработка неверного ввода

2.3 Соблюдение Google C++ Style Guide

При оформлении кода применялись правила руководства [1]:

- имена функций записываются в стиле PascalCase (FindFirst, ReadIntInRange), имена переменных и параметров – строчными буквами через подчёркивание (day_number, ins_size), имена констант – с префиксом k (kMaxSize);
- длина строки не превышает 80 символов, открывающая скобка – в конце строки;
- защита заголовка оформлена в виде LAB1_METHODS_H_, заголовочные файлы подключаются в порядке: свой заголовок, системные, стандартной библиотеки C++, остальные;
- перед объявлением каждой функции в заголовке есть комментарий о её назначении, а выходной параметр функции FindAll передаётся указателем и стоит после входных параметров;
- используются int64_t и nullptr.

Отступления от руководства: отступ равен стандартной табуляции Visual Studio – 4 пробела (в руководстве – 2); имена файлов сохранены ради совместимости с проектом; функции задания 4 по условию возвращают указатель на память из new[] (вместо std::vector), которую освобождает вызывающий код.

3 Задание 1. Методы

Первое задание посвящено базовой работе с типами данных: преобразованию вещественного числа в целое, символа в число, а также логическим выражениям с результатом типа `bool`.

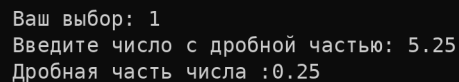
3.1 Задача 1. Дробная часть

Условие. Реализовать функцию `double fraction(double x)`, возвращающую дробную часть числа x (для $5,25 - 0,25$).

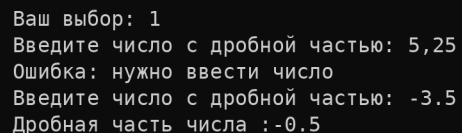
Решение. Приведение `static_cast<int64_t>(number)` [4] отбрасывает дробную часть (округляет к нулю), и разность `number` и целой части даёт дробную часть. Знак при этом сохраняется: для $-3,5$ получается $-0,5$. Тип `int64_t` выбран вместо `int`, чтобы корректно обрабатывать числа, большие 2^{31} .

Листинг 1 – Функция `Fraction`

```
// Задача 1 Дробная часть
double Fraction(double number) {
    return number - static_cast<int64_t>(number);
}
```



Ваш выбор: 1
Введите число с дробной частью: 5,25
Дробная часть числа :0.25



Ваш выбор: 1
Введите число с дробной частью: 5,25
Ошибка: нужно ввести число
Введите число с дробной частью: -3.5
Дробная часть числа :-0.5

Рисунок 2 – Работа функции `Fraction` и ввод числа с запятой

3.2 Задача 3. Букву в число

Условие. Реализовать функцию `int charToNum(char x)`, преобразующую символ-цифру «0»...«9» в соответствующее число.

Решение. Коды цифр в таблице символов идут подряд: код «0» равен 48, код «9» – 57. Поэтому разность кода символа и кода «0» равна значению цифры. Корректность ввода (ровно один символ-цифра) проверяет функция `ReadCharToNum`.

Листинг 2 – Функция CharToNum

```
// Задача 3 Букву в число
int CharToNum(char symbol) {
    return symbol - '0';
}
```

```
Ваш выбор: 2
Введите символ от 0 до 9: 3
Ваше число :3
```

```
Ваш выбор: 2
Введите символ от 0 до 9: а
Ошибка: нужно ввести один символ от 0 до 9
Введите символ от 0 до 9: 35
Ошибка: нужно ввести один символ от 0 до 9
Введите символ от 0 до 9: 7
Ваше число :7
```

Рисунок 3 – Работа функции CharToNum и проверка ввода символа

3.3 Задача 5. Двузначное

Условие. Реализовать функцию `bool is2Digits(int x)`, возвращающую `true`, если число двузначное (32 – `true`, 516 – `false`).

Решение. Число двузначное, если оно лежит в диапазоне от 10 до 99.

Результат проверки возвращается с помощью конструкции `if/else`.

Листинг 3 – Функция Is2Digits

```
// Задача 5 Двузначное
bool Is2Digits(int number) {
    if (10 <= number && number <= 99) {
        return true;
    } else {
        return false;
    }
}
```

```
Ваш выбор: 3
Введите число: 32
Двузначность числа :true (да)
```

```
Ваш выбор: 3
Введите число: 516
Двузначность числа :false (нет)
```

```
Ваш выбор: 3
Введите число: 4х
Ошибка: нужно ввести целое число
Введите число: 45
Двузначность числа :true (да)
```

Рисунок 4 – Работа функции Is2Digits

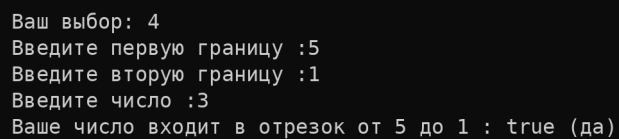
3.4 Задача 7. Диапазон

Условие. Реализовать функцию `bool isInRange(int a, int b, int num)`, возвращающую `true`, если `num` входит в диапазон между `a` и `b` включительно; какая из границ больше, заранее неизвестно.

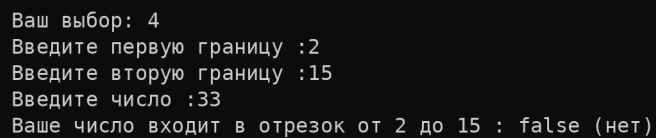
Решение. Так как порядок границ неизвестен, проверяются оба случая: $\text{limit_one} \leq \text{number} \leq \text{limit_two}$ или $\text{limit_two} \leq \text{number} \leq \text{limit_one}$. Нестрогие сравнения включают границы в диапазон.

Листинг 4 – Функция IsInRange

```
// Задача 7 Диапазон
bool IsInRange(int limit_one, int limit_two, int number) {
    if ((limit_one <= number && number <= limit_two) ||
        (limit_one >= number && number >= limit_two)) {
        return true;
    } else {
        return false;
    }
}
```



```
Ваш выбор: 4
Введите первую границу :5
Введите вторую границу :1
Введите число :3
Ваше число входит в отрезок от 5 до 1 : true (да)
```



```
Ваш выбор: 4
Введите первую границу :2
Введите вторую границу :15
Введите число :33
Ваше число входит в отрезок от 2 до 15 : false (нет)
```

Рисунок 5 – Работа функции IsInRange

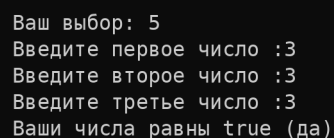
3.5 Задача 9. Равенство

Условие. Реализовать функцию `bool isEqual(int a, int b, int c)`, возвращающую `true`, если все три числа равны.

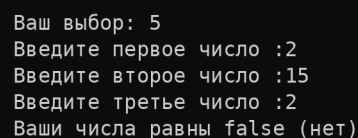
Решение. Равенство транзитивно, поэтому достаточно двух сравнений: `number_one == number_two` и `number_two == number_three`.

Листинг 5 – Функция IsEqual

```
// Задача 9 Равенство
bool IsEqual(int number_one, int number_two, int number_three) {
    if (number_one == number_two && number_two == number_three) {
        return true;
    } else {
        return false;
    }
}
```



```
Ваш выбор: 5
Введите первое число :3
Введите второе число :3
Введите третье число :3
Ваши числа равны true (да)
```



```
Ваш выбор: 5
Введите первое число :2
Введите второе число :15
Введите третье число :2
Ваши числа равны false (нет)
```

Рисунок 6 – Работа функции IsEqual

3.6 Результаты тестирования

Функции задания проверены автотестами на примерах из условия и на граничных случаях (таблица 2). Все тесты пройдены.

Таблица 2 – Результаты тестирования функций задания 1

№	Вызов	Ожидается	Получено	Итог
1	Fraction(5.25)	0.25	0.25	пройден
1	Fraction(-3.5)	-0.5	-0.5	пройден
1	Fraction(7.0)	0	0	пройден
3	CharToNum('3')	3	3	пройден
3	CharToNum('0')	0	0	пройден
3	CharToNum('9')	9	9	пройден
5	Is2Digits(32)	true	true	пройден
5	Is2Digits(516)	false	false	пройден
5	Is2Digits(-45)	false	false	пройден
5	Is2Digits(9)	false	false	пройден
7	IsInRange(5, 1, 3)	true	true	пройден
7	IsInRange(2, 15, 33)	false	false	пройден
7	IsInRange(2, 15, 15)	true	true	пройден
9	IsEqual(3, 3, 3)	true	true	пройден
9	IsEqual(2, 15, 2)	false	false	пройден
9	IsEqual(-1, -1, -1)	true	true	пройден

4 Задание 2. Условия

Второе задание обрабатывает ветвления: инструкции `if/else` и множественный выбор `switch`.

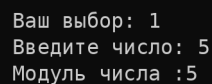
4.1 Задача 1. Модуль числа

Условие. Реализовать функцию `int abs(int x)`, возвращающую модуль числа `x`.

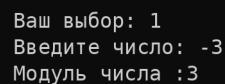
Решение. Если число отрицательное, функция возвращает его с противоположным знаком, иначе – само число. Имя `AbsNum` выбрано, чтобы не путать функцию со стандартной `std::abs`. Эта функция также используется в задаче «Поиск максимального» из задания 4.

Листинг 6 – Функция `AbsNum`

```
// Задача 1 Модуль числа
int AbsNum(int number) {
    if (number >= 0) {
        return number;
    } else {
        return -number;
    }
}
```



```
Ваш выбор: 1
Введите число: 5
Модуль числа :5
```



```
Ваш выбор: 1
Введите число: -3
Модуль числа :3
```

Рисунок 7 – Работа функции `AbsNum`

4.2 Задача 3. Тридцать пять

Условие. Реализовать функцию `bool is35(int x)`, возвращающую `true`, если число делится на 3 или на 5, но не на оба числа одновременно.

Решение. Результаты проверки остатков от деления (%) сохраняются в логические переменные `by3` и `by5`. Число подходит, если оно делится ровно на одно из чисел, то есть значения различны: `by3 != by5`.

Листинг 7 – Функция Is35

```
// Задача 3 Тридцать пять
bool Is35(int number) {
    bool by3 = (number % 3 == 0);
    bool by5 = (number % 5 == 0);
    if (by3 != by5) {
        return true;
    } else {
        return false;
    }
}
```

```
Ваш выбор: 2
Введите число: 5
Ваше число делится на 3 или 5, но не делится на них одновременно : true (да)
```

```
Ваш выбор: 2
Введите число: 8
Ваше число делится на 3 или 5, но не делится на них одновременно : false (нет)
```

```
Ваш выбор: 2
Введите число: 15
Ваше число делится на 3 или 5, но не делится на них одновременно : false (нет)
```

Рисунок 8 – Работа функции Is35

4.3 Задача 5. Тройной максимум

Условие. Реализовать функцию `int max3(int x, int y, int z)`, возвращающую максимальное из трёх чисел; идеальное решение содержит две инструкции `if` без вложенности.

Решение. Переменная `result` инициализируется значением `number_one`. Затем двумя последовательными `if` она заменяется на `number_two` или `number_three`, если они больше текущего максимума.

Листинг 8 – Функция Max3

```
// Задача 5 Тройной максимум
int Max3(int number_one, int number_two, int number_three) {
    int result = number_one;
    if (number_two > result) {
        result = number_two;
    }
    if (number_three > result) {
        result = number_three;
    }
    return result;
}
```

```
Ваш выбор: 3
Введите первое число :5
Введите второе число :7
Введите третье число :7
Максимальное число из трех :7
```

```
Ваш выбор: 3
Введите первое число :8
Введите второе число :-1
Введите третье число :4
Максимальное число из трех :8
```

Рисунок 9 – Работа функции Max3

4.4 Задача 7. Двойная сумма

Условие. Реализовать функцию `int sum2(int x, int y)`, возвращающую сумму `x` и `y`; если сумма попадает в диапазон от 10 до 19, вернуть 20.

Решение. Сумма вычисляется один раз и сохраняется в переменную, затем проверяется на попадание в отрезок `[10; 19]`.

Листинг 9 – Функция Sum2

```
// Задача 7 Двойная сумма
int Sum2(int number_one, int number_two) {
    int sum = number_one + number_two;
    if (sum >= 10 && sum <= 19) {
        return 20;
    }
    return sum;
}
```

```
Ваш выбор: 4
Введите первое число :5
Введите второе число :7
Сумма чисел :20
```

```
Ваш выбор: 4
Введите первое число :8
Введите второе число :-1
Сумма чисел :7
```

Рисунок 10 – Работа функции Sum2

4.5 Задача 9. День недели

Условие. Реализовать функцию `String day(int x)`, возвращающую название дня недели по его номеру (1 – понедельник, 7 – воскресенье); иначе вернуть «это не день недели». Использовать `switch`.

Решение. Каждому номеру дня соответствует своя ветвь `case` с возвратом строки. Все остальные значения обрабатывает ветвь `default`. Инструкция `break` не нужна, так как каждая ветвь завершается `return`.

Листинг 10 – Функция Day

```
// Задача 9 День недели
std::string Day(int day_number) {
    switch (day_number) {
        case 1:
            return "Понедельник";
        case 2:
            return "Вторник";
        case 3:
            return "Среда";
        case 4:
            return "Четверг";
        case 5:
            return "Пятница";
        case 6:
            return "Суббота";
        case 7:
            return "Воскресенье";
        default:
            return "Это не день недели";
    }
}
```

Ваш выбор: 5
Введите номер дня недели от 1 до 7 :5
День недели :Пятница

Ваш выбор: 5
Введите номер дня недели от 1 до 7 :18
День недели :Это не день недели

Рисунок 11 – Работа функции Day

4.6 Результаты тестирования

Результаты автотестов функций задания 2 приведены в таблице 3. Все тесты пройдены, включая граничные значения суммы 19 и 21.

Таблица 3 – Результаты тестирования функций задания 2

№	Вызов	Ожидается	Получено	Итог
1	AbsNum(5)	5	5	пройден
1	AbsNum(-3)	3	3	пройден
1	AbsNum(0)	0	0	пройден
3	Is35(5)	true	true	пройден
3	Is35(8)	false	false	пройден
3	Is35(15)	false	false	пройден
3	Is35(9)	true	true	пройден
5	Max3(5, 7, 7)	7	7	пройден
5	Max3(8, -1, 4)	8	8	пройден
5	Max3(-5, -2, -9)	-2	-2	пройден
7	Sum2(5, 7)	20	20	пройден
7	Sum2(8, -1)	7	7	пройден
7	Sum2(10, 9)	20	20	пройден
7	Sum2(10, 10)	20	20	пройден
7	Sum2(15, 6)	21	21	пройден
9	Day(5)	Пятница	Пятница	пройден
9	Day(1)	Понедельник	Понедельник	пройден
9	Day(7)	Воскресенье	Воскресенье	пройден
9	Day(18)	Это не день недели	Это не день недели	пройден

5 Задание 3. Циклы

Третье задание отрабатывает циклы `for`, `while` и `do-while`, в том числе вложенные циклы для вывода фигур из символов.

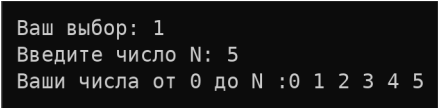
5.1 Задача 1. Числа подряд

Условие. Реализовать функцию `String listNums(int x)`, возвращающую строку со всеми числами от 0 до `x` включительно (для 5 – «0 1 2 3 4 5»).

Решение. Цикл `for` добавляет к строке каждое число от 0 до `limit` и пробел после него, а завершающий пробел затем удаляется методом `pop_back()`. Значение `N` при вводе ограничено отрезком 0...1000.

Листинг 11 – Функция `ListNums`

```
// Задача 1 Числа подряд
std::string ListNums(int limit) {
    std::string result = "";
    for (int i = 0; i <= limit; ++i) {
        result += std::to_string(i) + " ";
    }
    if (!result.empty()) {
        result.pop_back();
    }
    return result;
}
```



```
Ваш выбор: 1
Введите число N: 5
Ваши числа от 0 до N :0 1 2 3 4 5
```

Рисунок 12 – Работа функции `ListNums`

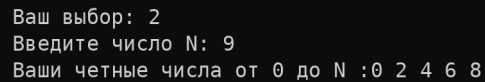
5.2 Задача 3. Чётные числа

Условие. Реализовать функцию `String chet(int x)`, возвращающую строку со всеми чётными числами от 0 до `x` включительно, не используя `if`.

Решение. Вместо проверки чётности цикл идёт с шагом 2, начиная с 2, поэтому в строку попадают только чётные числа. Инструкция `if` в функции не используется.

Листинг 12 – Функция Chet

```
// Задача 3 Чётные числа
std::string Chet(int limit) {
    std::string result = "0";
    for (int i = 2; i <= limit; i += 2) {
        result += " " + std::to_string(i);
    }
    return result;
}
```



```
Ваш выбор: 2
Введите число N: 9
Ваши четные числа от 0 до N :0 2 4 6 8
```

Рисунок 13 – Работа функции Chet

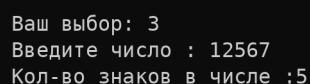
5.3 Задача 5. Длина числа

Условие. Реализовать функцию `int numLen(long x)`, возвращающую количество знаков в числе `x`.

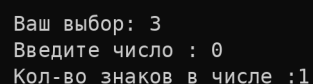
Решение. Для нуля функция сразу возвращает 1. Иначе в цикле `while` число делится на 10, пока не станет нулём, и считается количество делений. Для отрицательных чисел целочисленное деление тоже приводит к нулю. Вместо `long` используется `int64_t`, так как размер `long` в Windows и Linux различается.

Листинг 13 – Функция NumLen

```
// Задача 5 Длина числа
int NumLen(int64_t num) {
    if (num == 0) {
        return 1;
    } else {
        int count = 0;
        while (num != 0) {
            num = num / 10;
            ++count;
        }
        return count;
    }
}
```



```
Ваш выбор: 3
Введите число : 12567
Кол-во знаков в числе :5
```



```
Ваш выбор: 3
Введите число : 0
Кол-во знаков в числе :1
```

Рисунок 14 – Работа функции NumLen

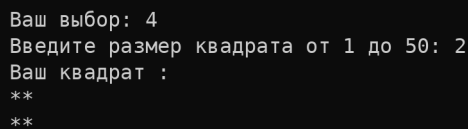
5.4 Задача 7. Квадрат

Условие. Реализовать функцию `void square(int x)`, выводящую квадрат из символов «*» размером $x \times x$.

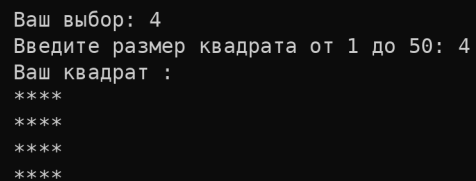
Решение. Внешний цикл перебирает строки, внутренний выводит x звёздочек, после чего выполняется перевод строки. Размер ограничен отрезком 1...50, чтобы фигура помещалась в окно консоли.

Листинг 14 – Функция Square

```
// Задача 7 Квадрат
void Square(int size) {
    for (int i = 0; i < size; ++i) {
        for (int j = 0; j < size; ++j) {
            std::cout << '*';
        }
        std::cout << '\n';
    }
}
```



```
Ваш выбор: 4
Введите размер квадрата от 1 до 50: 2
Ваш квадрат :
**
**
```



```
Ваш выбор: 4
Введите размер квадрата от 1 до 50: 4
Ваш квадрат :
****
****
****
****
```

Рисунок 15 – Работа функции Square при $x = 2$ и $x = 4$

5.5 Задача 9. Правый треугольник

Условие. Реализовать функцию `void rightTriangle(int x)`, выводящую треугольник из «*» высотой x , выровненный по правому краю.

Решение. В строке с номером i сначала выводится $\text{size} - i$ пробелов, затем i звёздочек. Пробелы сдвигают звёздочки к правому краю.

Листинг 15 – Функция RightTriangle

```
// Задача 9 Правый треугольник
void RightTriangle(int size) {
    for (int i = 1; i <= size; ++i) {
        for (int j = 0; j < size - i; ++j) {
            std::cout << ' ';
        }
        for (int j = 0; j < i; ++j) {
            std::cout << '*';
        }
        std::cout << '\n';
    }
}
```

```

Ваш выбор: 5
Введите размер треугольника от 1 до 50: 3
Ваш треугольник :
*
**
***

```

```

Ваш выбор: 5
Введите размер треугольника от 1 до 50: 4
Ваш треугольник :
*
**
***
****

```

Рисунок 16 – Работа функции RightTriangle при $x = 3$ и $x = 4$

5.6 Результаты тестирования

Результаты автотестов приведены в таблице 4. Вывод функций Square и RightTriangle перехватывался через буфер потока `std::cout` и сравнивался с эталонной строкой.

Таблица 4 – Результаты тестирования функций задания 3

№	Вызов	Ожидается	Получено	Итог
1	ListNums(5)	0 1 2 3 4 5	0 1 2 3 4 5	пройден
1	ListNums(0)	0	0	пройден
1	ListNums(-1)			пройден
3	Chet(9)	0 2 4 6 8	0 2 4 6 8	пройден
3	Chet(10)	0 2 4 6 8 10	0 2 4 6 8 10	пройден
5	NumLen(12567)	5	5	пройден
5	NumLen(0)	1	1	пройден
5	NumLen(-305)	3	3	пройден
7	Square(2)	** **	** **	пройден
7	Square(4)	**** **** **** ****	**** **** **** ****	пройден
9	RightTriangle(3)	* ** ***	* ** ***	пройден
9	RightTriangle(4)	* ** *** ****	* ** *** ****	пройден

6 Задание 4. Массивы

Четвёртое задание посвящено типовым алгоритмам на массивах: линейному поиску, поиску максимума, вставке, обращению и выборке элементов. В C++ массив, переданный в функцию, превращается в указатель, поэтому все функции дополнительно принимают размер массива. Функции, которые по условию возвращают «новый массив», выделяют память через `new[]`, а вызывающий код (меню Task4) освобождает её через `delete[]`. Массивы вводятся с клавиатуры поэлементно с проверкой каждого значения.

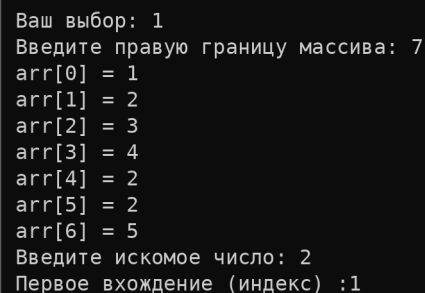
6.1 Задача 1. Поиск первого значения

Условие. Реализовать функцию `int findFirst(int arr[], int x)`, возвращающую индекс первого вхождения `x` в массив или `-1`.

Решение. Линейный поиск: массив просматривается с начала, и при первом совпадении функция сразу возвращает индекс. Если цикл закончился без совпадений, возвращается `-1`.

Листинг 16 – Функция FindFirst

```
// Задача 1 Поиск первого значения
int FindFirst(const int* arr, int number, int limit) {
    for (int i = 0; i < limit; ++i) {
        if (arr[i] == number) {
            return i;
        }
    }
    return -1;
}
```



```
Ваш выбор: 1
Введите правую границу массива: 7
arr[0] = 1
arr[1] = 2
arr[2] = 3
arr[3] = 4
arr[4] = 2
arr[5] = 2
arr[6] = 5
Введите искомое число: 2
Первое вхождение (индекс) :1
```

Рисунок 17 – Работа функции FindFirst

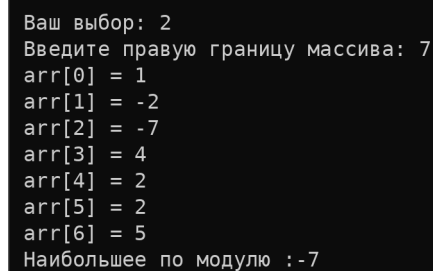
6.2 Задача 3. Поиск максимального

Условие. Реализовать функцию `int maxAbs(int arr[])`, возвращающую наибольшее по модулю значение массива (для `[1, -2, -7, 4, 2, 2, 5]` – `-7`).

Решение. Начальное значение результата – 0. Элементы сравниваются по модулю с помощью функции `AbsNum` из задания 2, и при большем модуле результат заменяется самим элементом со своим знаком.

Листинг 17 – Функция `MaxAbs`

```
// Задача 3 Поиск максимального по модулю
int MaxAbs(const int* arr, int limit) {
    int result = 0;
    for (int i = 0; i < limit; ++i) {
        if (AbsNum(arr[i]) > AbsNum(result)) {
            result = arr[i];
        }
    }
    return result;
}
```



```
Ваш выбор: 2
Введите правую границу массива: 7
arr[0] = 1
arr[1] = -2
arr[2] = -7
arr[3] = 4
arr[4] = 2
arr[5] = 2
arr[6] = 5
Наибольшее по модулю :-7
```

Рисунок 18 – Работа функции `MaxAbs`

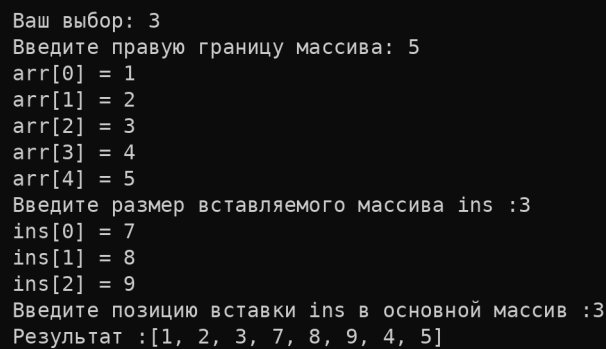
6.3 Задача 5. Добавление массива в массив

Условие. Реализовать функцию `int* add(int arr[], int ins[], int pos)`, возвращающую новый массив, в котором в позицию `pos` вставлены элементы `ins`.

Решение. Сначала проверяются параметры: при отрицательных размерах или позиции вне отрезка `[0; limit_one]` возвращается `nullptr`. Затем выделяется массив длины `limit_one + limit_two`, и тремя циклами в него копируются начало `arr`, весь `ins` и остаток `arr`. Позицию записи задаёт счётчик `limit_result`.

Листинг 18 – Функция Add

```
// Задача 5 Добавление массива в массив
int* Add(const int* arr, const int* ins, int pos, int limit_one,
        int limit_two) {
    if (pos < 0 || pos > limit_one || limit_one < 0 || limit_two < 0) {
        return nullptr;
    }
    int* result = new int[limit_one + limit_two];
    int limit_result = 0;
    for (int i = 0; i < pos; ++i) {
        result[limit_result] = arr[i];
        ++limit_result;
    }
    for (int i = 0; i < limit_two; ++i) {
        result[limit_result] = ins[i];
        ++limit_result;
    }
    for (int i = pos; i < limit_one; ++i) {
        result[limit_result] = arr[i];
        ++limit_result;
    }
    return result;
}
```



```
Ваш выбор: 3
Введите правую границу массива: 5
arr[0] = 1
arr[1] = 2
arr[2] = 3
arr[3] = 4
arr[4] = 5
Введите размер вставляемого массива ins :3
ins[0] = 7
ins[1] = 8
ins[2] = 9
Введите позицию вставки ins в основной массив :3
Результат :[1, 2, 3, 7, 8, 9, 4, 5]
```

Рисунок 19 – Работа функции Add

6.4 Задача 7. Возвратный реверс

Условие. Реализовать функцию `int* reverseBack(int arr[])`, возвращающую новый массив, в котором элементы `arr` записаны задом наперёд.

Решение. Цикл проходит исходный массив с конца, а счётчик `limit_result` задаёт позицию в новом массиве. Исходный массив не изменяется, так как передаётся как `const int*`.

Листинг 19 – Функция ReverseBack

```
// Задача 7 Возвратный реверс
int* ReverseBack(const int* arr, int limit) {
    int* result = new int[limit];
    int limit_result = 0;
    for (int i = limit - 1; i >= 0; --i) {
        result[limit_result] = arr[i];
        ++limit_result;
    }
    return result;
}
```

```
Ваш выбор: 4
Введите правую границу массива: 5
arr[0] = 1
arr[1] = 2
arr[2] = 3
arr[3] = 4
arr[4] = 5
Результат :[5, 4, 3, 2, 1]
```

Рисунок 20 – Работа функции ReverseBack

6.5 Задача 9. Все вхождения

Условие. Реализовать функцию `int* findAll(int arr[], int x)`, возвращающую новый массив индексов всех вхождений `x` в `arr`.

Решение. Задача решается в два прохода. Первый проход считает число вхождений, чтобы выделить массив нужного размера, второй записывает в него индексы. Количество найденных индексов возвращается через параметр-указатель `count_limit`. Если вхождений нет, меню выводит «Число не найдено в массиве».

Листинг 20 – Функция FindAll

```
// Задача 9 Все вхождения
int* FindAll(const int* arr, int number, int limit, int* count_limit) {
    *count_limit = 0;
    for (int i = 0; i < limit; ++i) {
        if (arr[i] == number) {
            ++*count_limit;
        }
    }
    int* result = new int[*count_limit];
    int limit_result = 0;
    for (int i = 0; i < limit; ++i) {
        if (arr[i] == number) {
            result[limit_result] = i;
            ++limit_result;
        }
    }
    return result;
}
```

```

Ваш выбор: 5
Введите правую границу массива: 7
arr[0] = 1
arr[1] = 2
arr[2] = 3
arr[3] = 8
arr[4] = 2
arr[5] = 2
arr[6] = 9
Введите искомое число, которое надо найти :2
Индексы вхождений :[1, 4, 5]

```

Рисунок 21 – Работа функции FindAll

6.6 Результаты тестирования

Результаты автотестов приведены в таблице 5. Тесты собирались с ключами `-fsanitize=address,undefined`: ошибок работы с памятью (выход за границы, утечки) не обнаружено.

Таблица 5 – Результаты тестирования функций задания 4

№	Вызов	Ожидается	Получено	Итог
1	FindFirst([1, 2, 3, 4, 2, 2, 5], 2)	1	1	пройден
1	FindFirst([1, 2, 3, 4, 2, 2, 5], 7)	-1	-1	пройден
3	MaxAbs([1, -2, -7, 4, 2, 2, 5])	-7	-7	пройден
5	Add([1, 2, 3, 4, 5], [7, 8, 9], 3)	[1,2,3,7,8,9,4,5]	[1,2,3,7,8,9,4,5]	пройден
5	Add([1, 2, 3, 4, 5], [7, 8, 9], 0)	[7,8,9,1,2,3,4,5]	[7,8,9,1,2,3,4,5]	пройден
5	Add(..., pos=9) (pos вне массива)	nullptr	nullptr	пройден
7	ReverseBack([1, 2, 3, 4, 5])	[5,4,3,2,1]	[5,4,3,2,1]	пройден
9	FindAll([1, 2, 3, 8, 2, 2, 9], 2)	[1,4,5]	[1,4,5]	пройден
9	FindAll([1, 2, 3, 8, 2, 2, 9], 6)	[]	[]	пройден

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы реализованы 20 функций: по пять задач (1, 3, 5, 7, 9) из каждого из четырёх заданий – «Методы», «Условия», «Циклы» и «Массивы». Все функции вызываются из двухуровневого меню в функции `main`. Ввод с клавиатуры проверяется: некорректные строки отклоняются, значения запрашиваются повторно, выход за допустимые границы контролируется.

Функции проверены 56 автотестами на примерах из условия и на граничных случаях: отрицательные числа, ноль, границы диапазонов, отсутствие искомого элемента, вставка в начало массива. Все тесты пройдены, а проверка санитайзерами не выявила ошибок работы с динамической памятью. При тестировании была найдена и исправлена ошибка проверки ввода: строки вида «35» или «12abc» раньше принимались частично (как «3» и «12»). Теперь строка разбирается целиком и принимается только тогда, когда в ней записано ровно одно значение.

Исходный код оформлен по руководству Google C++ Style Guide: единые правила именования, длина строк до 80 символов, отступ в 4 пробела, комментарии к объявлениям и защита заголовка. Утилита `crplint` замечаний не выдала. В результате работы закрепились навыки работы с типами данных, ветвлениями, циклами, массивами и динамической памятью в языке C++.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Google C++ Style Guide : [сайт]. – URL: <https://google.github.io/styleguide/cppguide.html> (дата обращения: 08.10.2026). – Текст : электронный.
2. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления : национальный стандарт Российской Федерации : издание официальное : введён впервые 2018-07-01. – Москва : Стандартинформ, 2017. – 32 с. – Текст : непосредственный.
3. Страуструп, Б. Программирование: принципы и практика с использованием C++ / Б. Страуструп ; пер. с англ. – 2-е изд. – Москва : Вильямс, 2016. – 1328 с. – Текст : непосредственный.
4. Справочник по C++ : [сайт] // cppreference.com. – URL: <https://ru.cppreference.com/w/cpp> (дата обращения: 08.10.2026). – Текст : электронный.
5. `cpplint` – Static code checker for C++ : [репозиторий] // GitHub. – URL: <https://github.com/cpplint/cpplint> (дата обращения: 08.10.2026). – Текст : электронный.

ПРИЛОЖЕНИЕ А

Исходный код программы

Листинг 21 – Файл Methods.h

```
#ifndef LAB1_METHODS_H_
#define LAB1_METHODS_H_

#include <cstdint>
#include <string>

// Максимальный размер массива, вводимого с клавиатуры
constexpr int kMaxSize = 100;

// Задание 1 Методы

// Возвращает дробную часть числа
double Fraction(double number);

// Преобразует символ-цифру от '0' до '9' в соответствующее число
int CharToNum(char symbol);

// Возвращает true, если число двузначное
bool Is2Digits(int number);

// Возвращает true, если number лежит между limit_one и limit_two
// включительно
// Какая из границ больше, заранее неизвестно
bool IsInRange(int limit_one, int limit_two, int number);

// Возвращает true, если все три числа равны
bool IsEqual(int number_one, int number_two, int number_three);

// Задание 2 Условия

// Возвращает модуль числа
int AbsNum(int number);

// Возвращает true, если число делится на 3 или на 5, но не на оба сразу
bool Is35(int number);

// Возвращает максимальное из трёх чисел
int Max3(int number_one, int number_two, int number_three);

// Возвращает сумму чисел, а если она лежит в диапазоне от 10 до 19, то 20
int Sum2(int number_one, int number_two);

// Возвращает название дня недели по номеру от 1 до 7
std::string Day(int day_number);

// Задание 3 Циклы

// Возвращает строку со всеми числами от 0 до limit включительно
std::string ListNums(int limit);

// Возвращает строку со всеми чётными числами от 0 до limit включительно
// Требуется limit >= 0
std::string Chet(int limit);

// Возвращает количество цифр в числе
int NumLen(int64_t num);

// Выводит на экран квадрат из символов '*' размером size x size
void Square(int size);

// Выводит на экран прямоугольный треугольник из '*',
// выровненный по правому краю
void RightTriangle(int size);

// Задание 4 Массивы

// Возвращает индекс первого вхождения number в массив или -1
int FindFirst(const int* arr, int number, int limit);

// Возвращает наибольшее по модулю значение массива
```

```

int MaxAbs(const int* arr, int limit);

// Возвращает новый массив размера limit_one + limit_two, в котором
// в позицию pos вставлены элементы массива ins
// При некорректных параметрах возвращает nullptr
// Память освобождает вызывающий
int* Add(const int* arr, const int* ins, int pos, int limit_one,
        int limit_two);

// Возвращает новый массив, в котором элементы arr записаны в обратном
// порядке
// Память освобождает вызывающий
int* ReverseBack(const int* arr, int limit);

// Возвращает новый массив индексов всех вхождений number в arr;
// количество найденных индексов записывается в *count_limit
// Память освобождает вызывающий
int* FindAll(const int* arr, int number, int limit, int* count_limit);

// Ввод с проверкой и вывод

// Выводит приглашение и считывает строку целиком
std::string ReadLine(const std::string& message);

// Запрашивает целое число до тех пор, пока ввод не будет корректным
int ReadInt(const std::string& message);

// Запрашивает вещественное число до тех пор, пока ввод не будет корректным
double ReadDouble(const std::string& message);

// Запрашивает один символ-цифру от '0' до '9'
char ReadCharToNum(const std::string& message);

// Запрашивает целое число из отрезка [min_value, max_value]
int ReadIntInRange(const std::string& message, int min_value, int max_value);

// Заполняет массив arr размера size значениями с клавиатуры
void ReadArr(int* arr, int size, const std::string& name);

// Выводит массив в виде [1, 2, 3]
void PrintArr(const int* arr, int size);

// Возвращает текстовое представление логического значения
std::string BoolText(bool value);

// Меню заданий

void Task1();
void Task2();
void Task3();
void Task4();

#endif // LAB1_METHODS_H_

```

Листинг 22 – Файл main.cpp

```

#ifdef _WIN32
#include <windows.h>
#endif

#include <locale>
#include <iostream>

#include "Methods.h"

int main() {
    std::setlocale(LC_ALL, "ru_RU.UTF-8");
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
#endif

    while (true) {
        std::cout << "\nРазделы лабораторной №1 \n";
        std::cout << "Выберите Задание из лабораторной №1 \n";
        std::cout << "Задание 1: Методы \n";
        std::cout << "Задание 2: Условия \n";
        std::cout << "Задание 3: Циклы \n";
    }
}

```

```

std::cout << "Задание 4: Массивы \n";
std::cout << "Выход: 0 \n";
int chose_task = ReadInt("Ваш выбор :");

switch (chose_task) {
    case 1:
        Task1();
        break;
    case 2:
        Task2();
        break;
    case 3:
        Task3();
        break;
    case 4:
        Task4();
        break;
    case 0:
        return 0;
    default:
        std::cout << "Такого задания нет \n";
        break;
}
}
}

```

Листинг 23 – Функции ввода с проверкой из файла Methods.cpp

```

// Если поток ввода закрыт, завершает программу
std::string ReadLine(const std::string& message) {
    std::cout << message;
    std::string line;
    if (!std::getline(std::cin, line)) {
        std::cout << "Ввод завершён \n";
        std::exit(0);
    }
    return line;
}

// Строка корректна, если в ней только целое число без лишних символов
int ReadInt(const std::string& message) {
    while (true) {
        std::stringstream stream(ReadLine(message));
        int value = 0;
        char extra = '\0';
        if (stream >> value && !(stream >> extra)) {
            return value;
        }
        std::cout << "Ошибка: нужно ввести целое число \n";
    }
}

// Строка корректна, если в ней только вещественное число без лишних
// символов
double ReadDouble(const std::string& message) {
    while (true) {
        std::stringstream stream(ReadLine(message));
        double value = 0;
        char extra = '\0';
        if (stream >> value && !(stream >> extra)) {
            return value;
        }
        std::cout << "Ошибка: нужно ввести число \n";
    }
}

// Принимается ровно один символ от '0' до '9'
char ReadCharToNum(const std::string& message) {
    while (true) {
        std::string line = ReadLine(message);
        if (line.size() == 1 && line[0] >= '0' && line[0] <= '9') {
            return line[0];
        }
        std::cout << "Ошибка: нужно ввести один символ от 0 до 9 \n";
    }
}

// Повторяет ввод, пока число не попадёт в отрезок от min_value до max_value

```

```

int ReadIntInRange(const std::string& message, int min_value, int max_value) {
    while (true) {
        int value = ReadInt(message);
        if (value >= min_value && value <= max_value) {
            return value;
        }
        std::cout << "Ошибка: число должно быть от " << min_value << " до "
            << max_value << " \n";
    }
}

// Ввод массива с клавиатуры с проверкой каждого элемента через ReadInt
void ReadArr(int* arr, int size, const std::string& name) {
    for (int i = 0; i < size; ++i) {
        arr[i] = ReadInt(name + "[" + std::to_string(i) + "] = ");
    }
}

// Вывод элементов массива через запятую в квадратных скобках
void PrintArr(const int* arr, int size) {
    std::cout << "[";
    for (int i = 0; i < size; ++i) {
        if (i > 0) {
            std::cout << ", ";
        }
        std::cout << arr[i];
    }
    std::cout << "]\n";
}

// Перевод логического значения в текст для вывода на экран
std::string BoolText(bool value) {
    return value ? "true (да)" : "false (нет)";
}

```